Atelier 7 : GitLab CI/CD + SonarQube Cloud

Rapport de travaux pratiques

Étudiant : Houssam Marmoud

Projet : atelier-sonarqube

Technologies : HTML, CSS, JavaScript, GitLab CI/CD, SonarQube Cloud, GitLab Pages

1. Objectif

Mettre en place un pipeline GitLab CI/CD qui analyse automatiquement un mini-site web avec SonarQube Cloud, puis déploie le site sur GitLab Pages uniquement si le Quality Gate est respecté. Deux versions du projet sont utilisées :

- V1 : code correct.
- V2 : code volontairement vulnérable (faille XSS).

2. Présentation des outils

- GitLab CI/CD : automatise les étapes (analyse, déploiement) à chaque git push.
- SonarQube Cloud : analyse la qualité et la sécurité du code (bugs, vulnérabilités, Security Hotspots, Code Smells).
- Quality Gate : ensemble de conditions qui détermine si le code satisfait les exigences de qualité du projet.

3. Configuration

1. Création du projet public atelier-sonarqube sur GitLab et clonage en local avec git clone.
2. Import de l'organisation SonarQube Cloud (4phtt) depuis GitLab, puis création du projet 4phtt_atelier-sonarqube.
3. Désactivation de l'Automatic Analysis, qui entre en conflit avec l'analyse lancée par le CI.
4. Création du fichier sonar-project.properties.
5. Création d'un jeton SonarQube et ajout des variables CI/CD SONAR_HOST_URL et SONAR_TOKEN dans GitLab.
6. Création du fichier .gitlab-ci.yml (jobs sonarqube-check et deploy-pages pour la V1).

Fichier sonar-project.properties :

```
sonar.projectKey=4phtt_atelier-sonarqube
sonar.organization=4phtt
sonar.projectName=Atelier CI CD SonarCloud

sonar.sources=.
sonar.sourceEncoding=UTF-8
sonar.exclusions=public/**

sonar.qualitygate.wait=true
```

4. Version 1 : code correct

4.1 Pipeline réussi

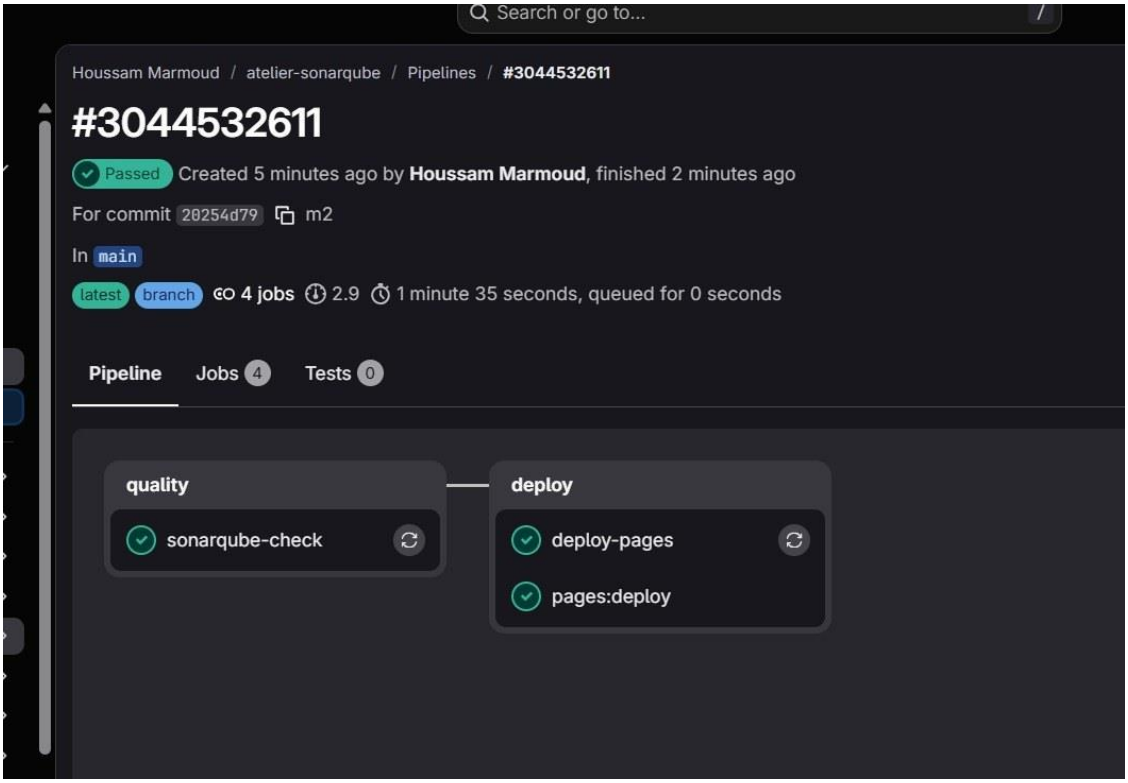


Figure 1 : pipeline V1 (#3044532611), jobs sonarqube-check et deploy-pages réussis

Les deux jobs réussissent : le code est analysé par SonarQube puis déployé.

4.2 Dépôt GitLab

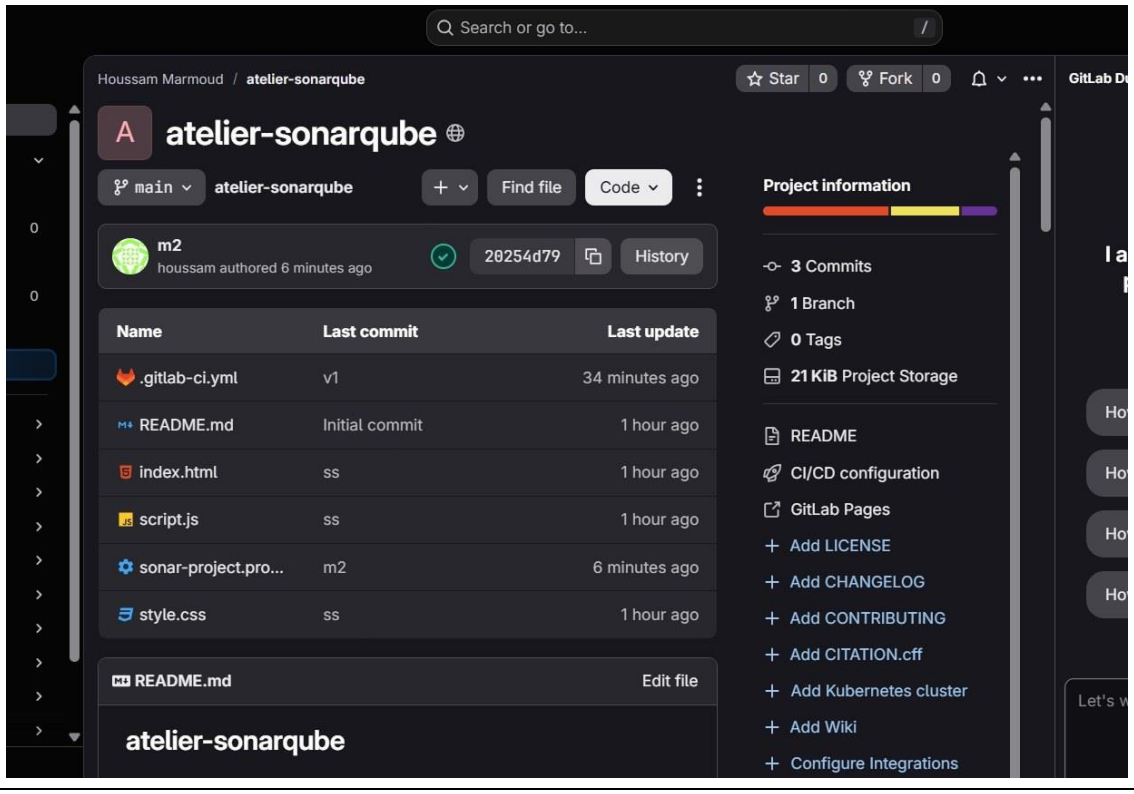


Figure 2 : commit m2 validé (coche verte) avec les fichiers du projet

Une première exécution avait échoué avec l'erreur « Not authorized or organization not found » : les valeurs sonar.projectKey et sonar.organization du fichier sonar-project.properties ne correspondaient pas à celles du projet SonarQube. Après correction (4phtt_atelier-sonarqube et 4phtt), le pipeline réussit.

4.3 Résultat SonarQube

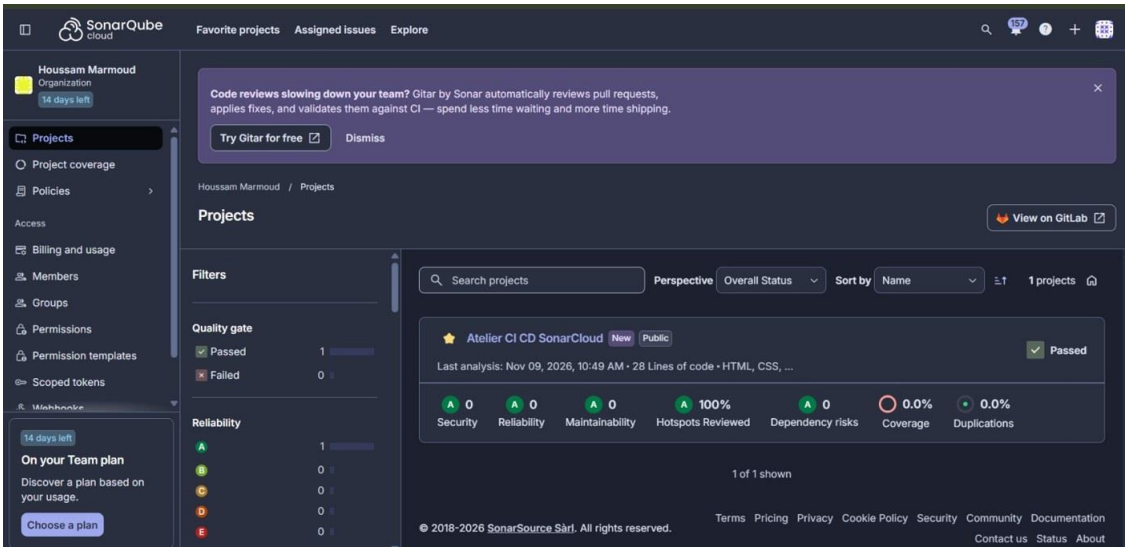


Figure 3 : Quality Gate « Passed » (Security A, Reliability A, Maintainability A)

Aucun problème n'est détecté : le Quality Gate est respecté.

4.4 Site déployé sur GitLab Pages



Figure 4 : site publié (« Version 1 : code correct »)

5. Version 2 : code vulnérable

Le code est modifié pour lire un paramètre de l'URL et l'insérer dans la page avec innerHTML :

```
const params = new URLSearchParams(window.location.search);
const value = params.get("message");

if (value) {
  document.getElementById("message").innerHTML = value;
}
```

Le fichier .gitlab-ci.yml est enrichi avec trois stages : quality, security et deploy (jobs syntax-check, sonarqube-check, security-check et deploy-pages).

5.1 Dépôt GitLab après le push

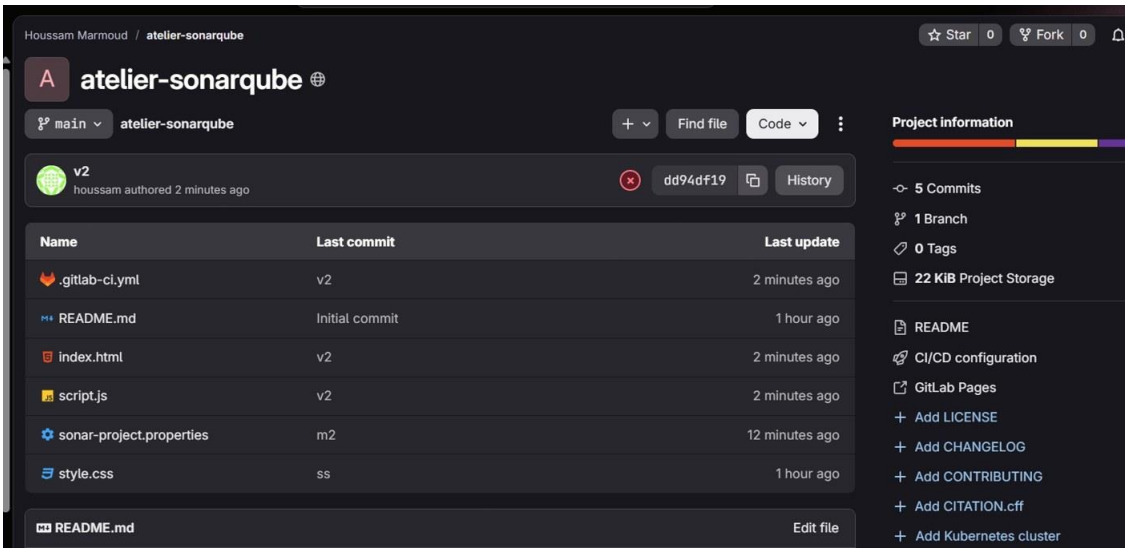


Figure 5 : commit v2 en échec (croix rouge)

5.2 Pipeline échoué

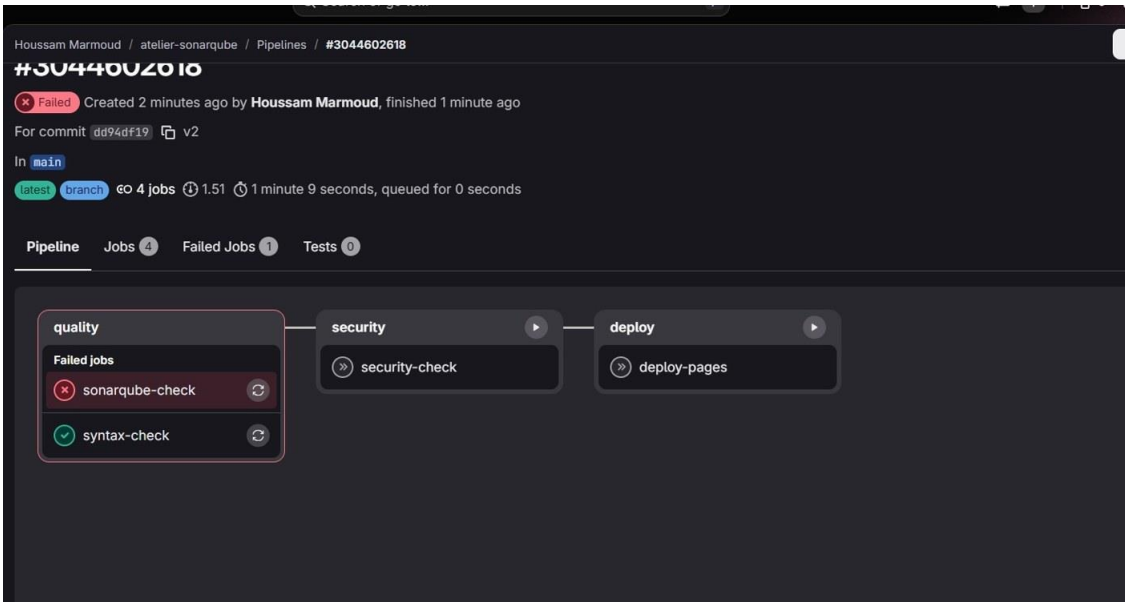


Figure 6 : pipeline V2 (#3044602618), sonarqube-check en échec, security-check et deploy-pages non exécutés

Le job sonarqube-check échoue, donc les stages suivants ne s'exécutent pas : le déploiement est bloqué.

5.3 Log du job sonarqube-check

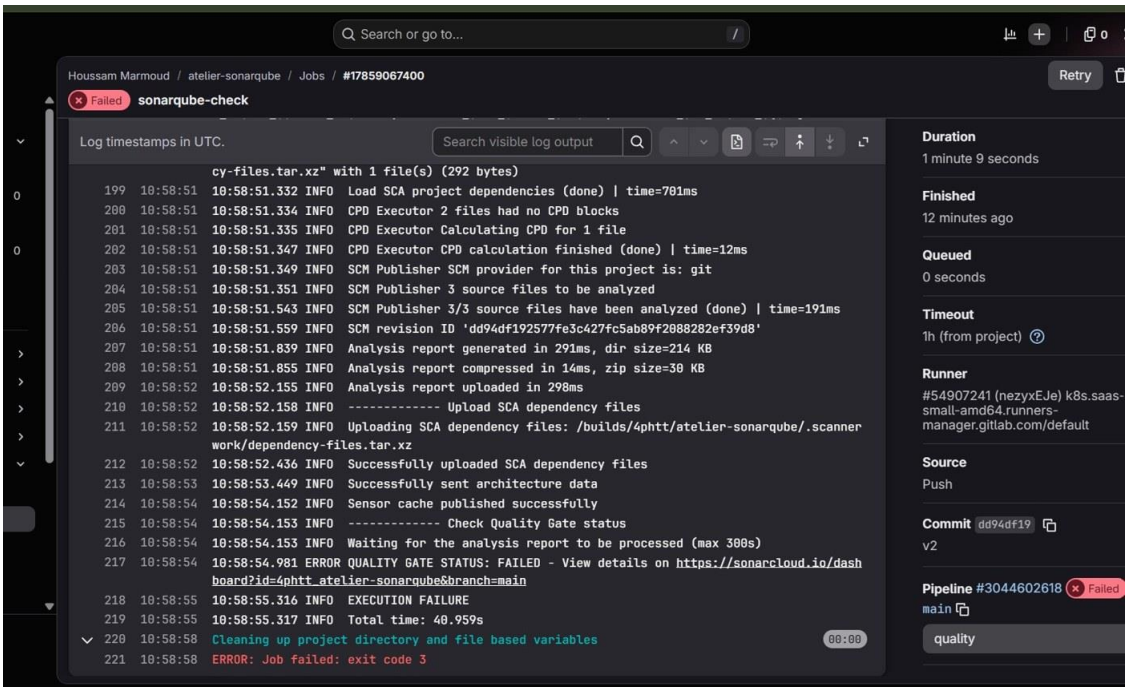


Figure 7 : « QUALITY GATE STATUS: FAILED » et « exit code 3 »

Grâce à sonar.qualitygate.wait=true, le job attend le verdict de SonarQube et échoue lorsque le Quality Gate n'est pas respecté.

5.4 Résultat SonarQube

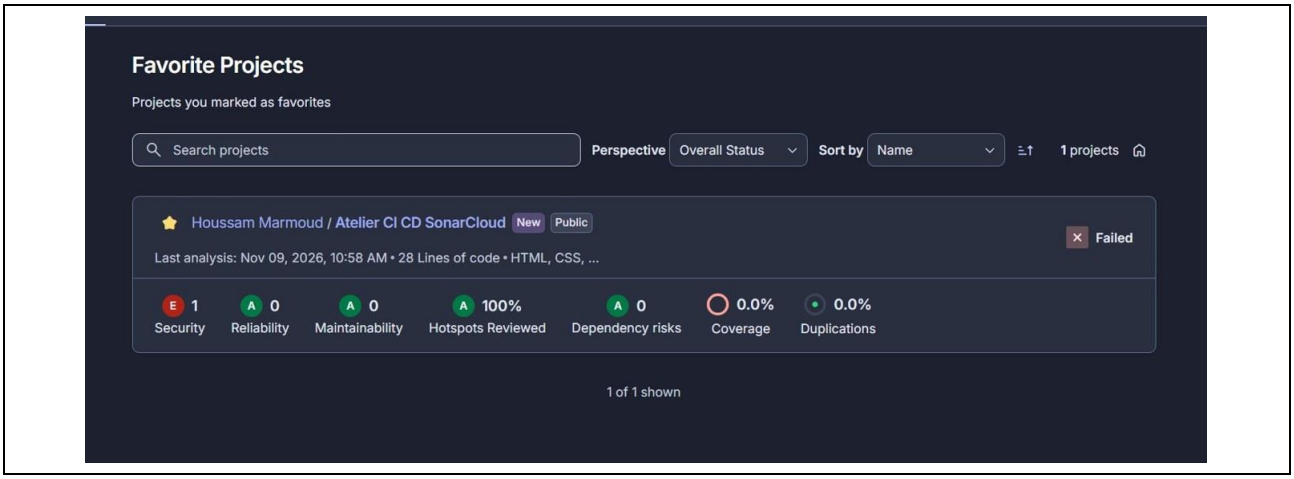


Figure 8 : Quality Gate « Failed », Security E avec 1 problème

5.5 Détail de la vulnérabilité

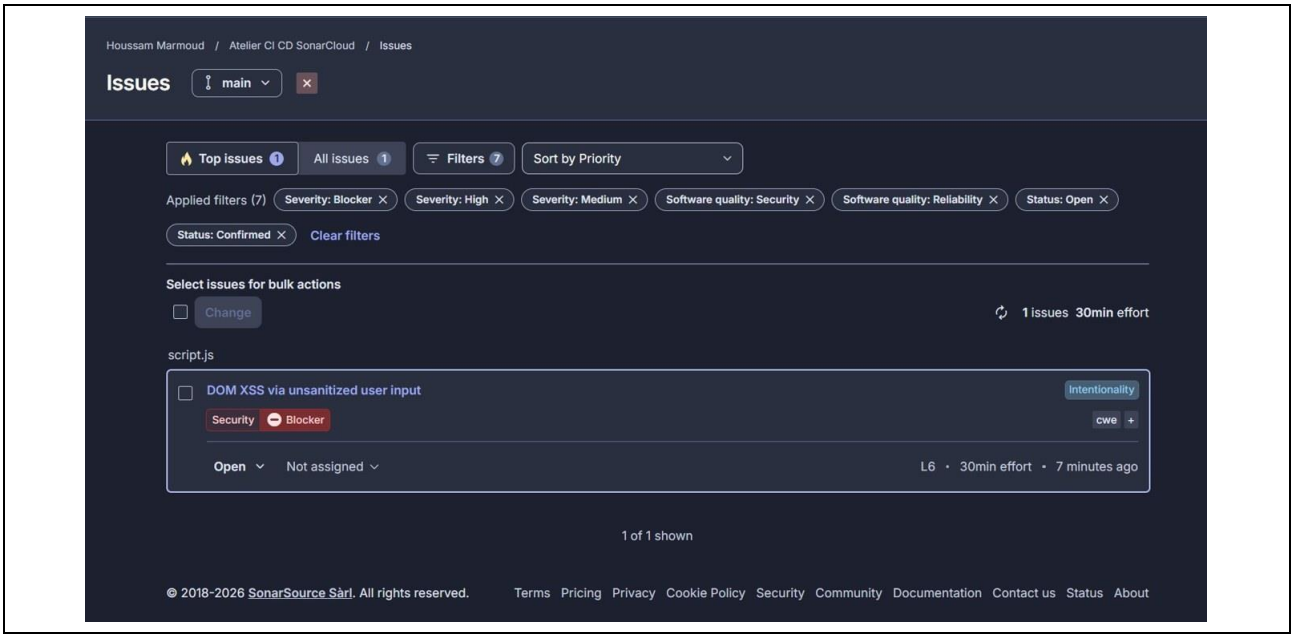


Figure 9 : DOM XSS via unsanitized user input, Blocker, script.js ligne 6

La donnée provenant de l'URL est insérée dans le DOM via innerHTML sans nettoyage : un attaquant peut injecter du code JavaScript.

6. Correction proposée

Remplacer innerHTML par textContent, qui traite la saisie comme du texte et non comme du HTML :

```
document.getElementById("message").textContent = value;
```

Après ce correctif et un nouveau git push, le pipeline doit réussir et le Quality Gate repasser à « Passed ».

7. Conclusion

1. Le développeur pousse son code sur GitLab.
2. GitLab déclenche automatiquement le pipeline CI/CD.

3. SonarQube Cloud analyse le code et détecte la faille XSS.
4. Le Quality Gate échoue et le job sonarqube-check échoue.
5. Le déploiement est bloqué : le code vulnérable n'atteint jamais le site.
6. Le développeur corrige, pousse à nouveau, et le cycle recommence.

À retenir : GitLab CI/CD automatise les étapes, SonarQube contrôle la qualité et la sécurité du code, et le Quality Gate empêche le déploiement d'un code qui ne respecte pas les critères. SonarQube ne remplace ni les tests, ni la revue de code.